

Building RAG Systems with LangChain and PostgreSQL

A comprehensive guide to implementing Retrieval-Augmented Generation systems using LangChain for document processing and PostgreSQL with pgvector for efficient vector similarity search



Document Loading and Preprocessing Pipeline



Document Loaders

PDF, DOCX, TXT, and CSV file formats supported



Metadata Extraction

Automatic metadata capture and enrichment



Text Cleaning

Preprocessing and normalization

LangChain provides robust document loaders that extract raw text content from various file formats while preserving structural metadata. The preprocessing pipeline handles encoding issues, removes noise, and normalizes text for downstream processing.

Advanced Text Chunking Strategies

Fixed-Size Chunking

Simple, predictable splits at character or token boundaries

Semantic Chunking

Intelligent splits based on document structure and meaning

Recursive Chunking

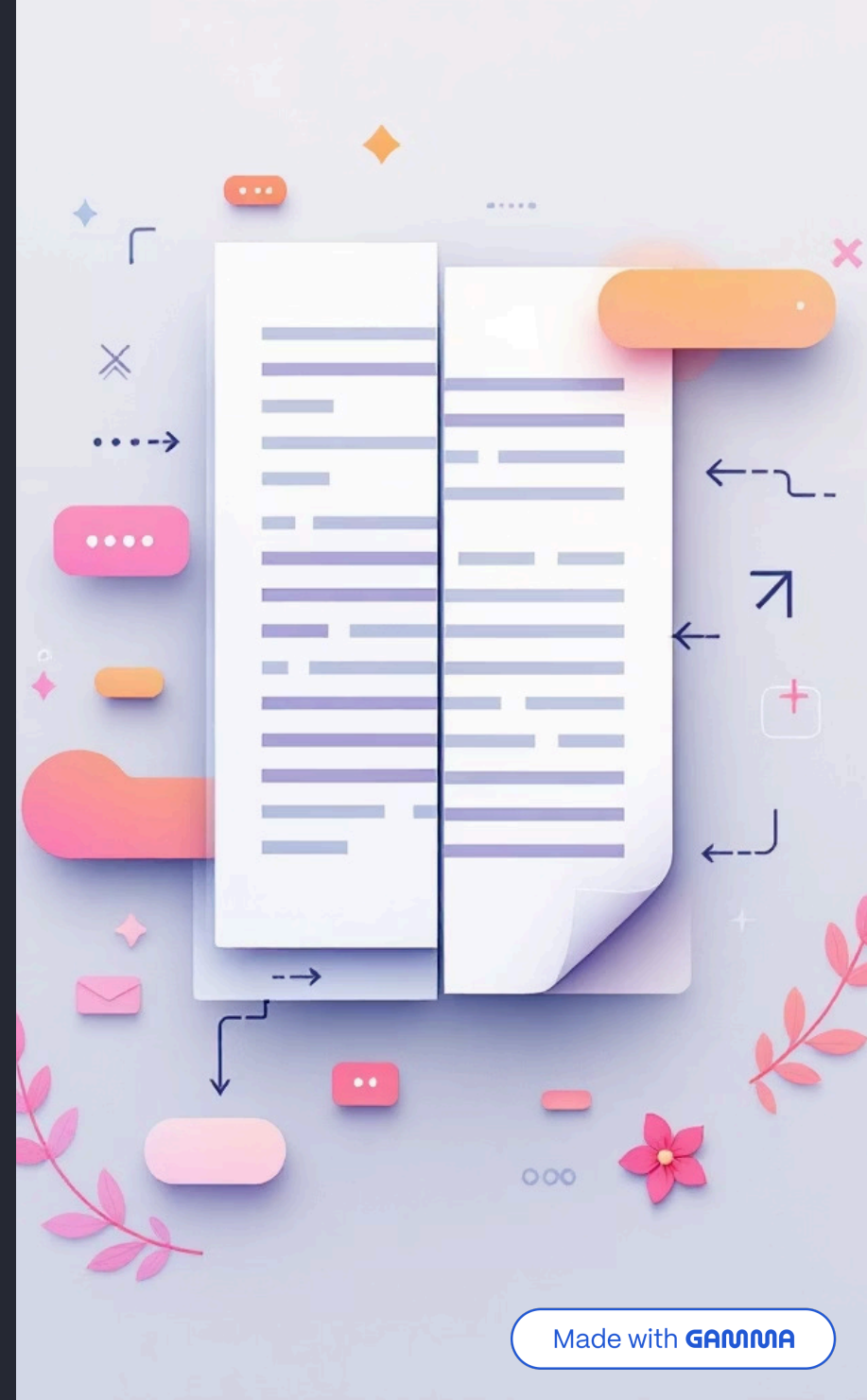
Multi-level splitting for hierarchical document organization

Chunk Overlap

Maintain context continuity by overlapping adjacent chunks by 10-20% of their size

Optimal Chunk Size

Balance between context richness
(512-1024 tokens) and retrieval
precision



Chunking Strategies - Explained

Fixed Size Chunking

- Splits text into equal-sized chunks (by tokens/characters)
- No understanding of meaning or structure
-  Fast, simple, scalable
-  May break sentences → loss of context
- **Best for:** large raw data, logs, quick pipelines

Semantic Chunking

- Splits text based on meaning and topic boundaries
- Groups related sentences/paragraphs together
-  High context preservation → better retrieval
-  Slower, requires embeddings/NLP
- **Best for:** FAQs, documentation, knowledge bases

Recursive Chunking

- Multi-step splitting strategy: Paragraph → Sentence → Word → Token
- Ensures chunks stay within size limits while preserving meaning
-  Balanced approach (structure + size control)
-  Most widely used in production
-  Slightly more complex to configure
- **Best for:** real-world RAG systems, LLM pipelines

Fixed: Fast but less accurate

Semantic: Accurate but slower

Recursive: Best balance (recommended)

Chunk Size and Chunk Overlap

(LangChain example)

```
RecursiveCharacterTextSplitter(  
    chunk_size=500, # characters  
    chunk_overlap=100 # characters  
)
```

```
RecursiveCharacterTextSplitter.from_tiktoken_encoder(  
    chunk_size=500, # tokens  
    chunk_overlap=100 #tokens  
)
```

PostgreSQL pgvector Extension



Extension Setup

Install pgvector extension in PostgreSQL 12+ using CREATE EXTENSION command



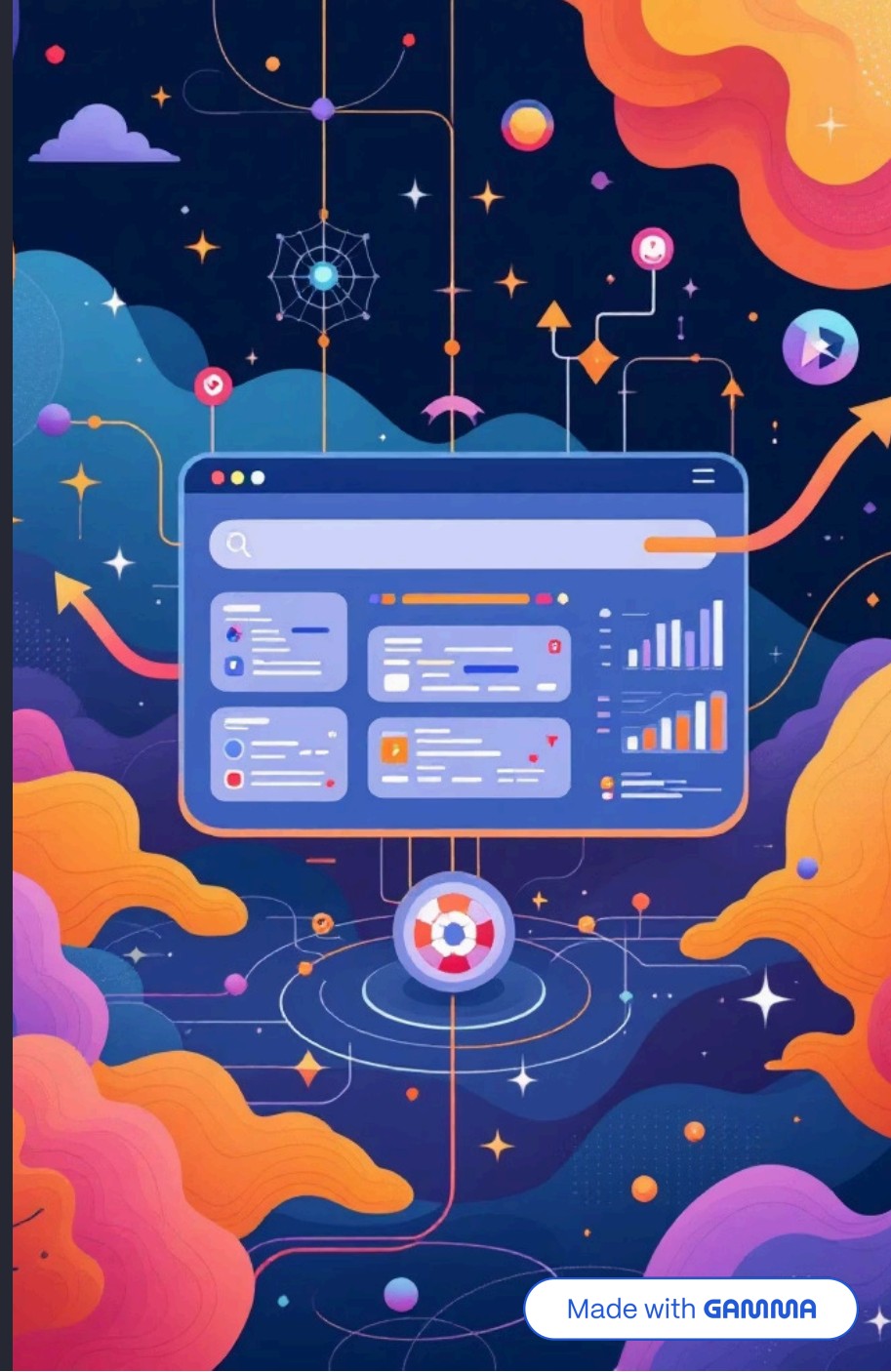
Vector Table Creation

Define tables with vector columns using VECTOR data type for embeddings storage



LangChain Integration

Connect using PGVector vector store for seamless document indexing and retrieval



Vector Indexing and Similarity Search

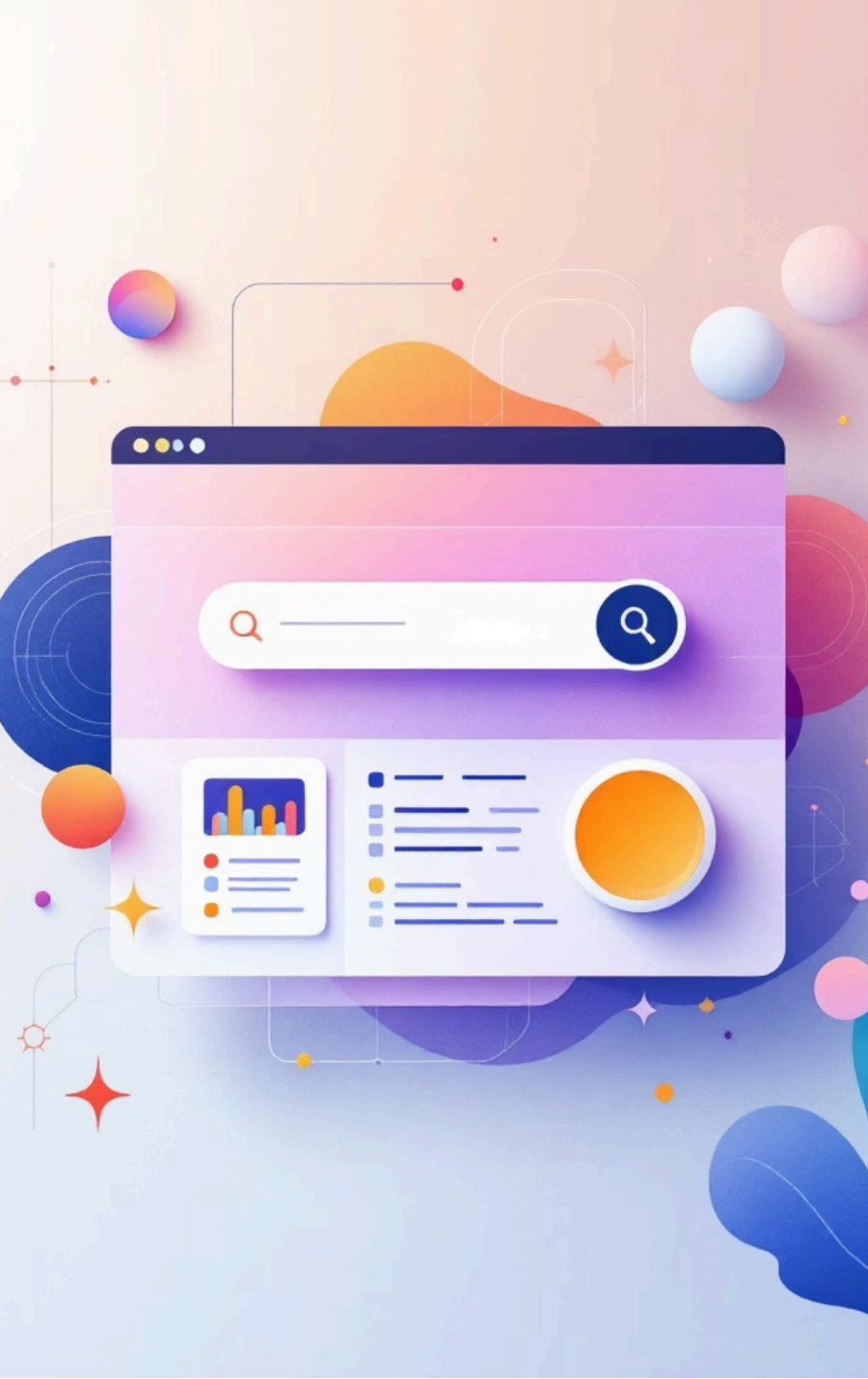
Document Indexing

Generate embeddings using models like OpenAI, BGE, or Sentence Transformers. Store vectors alongside document content and metadata in PostgreSQL tables.

Vector Similarity Queries

- Calculate cosine similarity between query and document vectors
- Use HNSW or IVFFlat indexes for efficient approximate nearest neighbor search
- Retrieve top-K most relevant documents





Hybrid Search: Best of Both Worlds

Vector Similarity Search

Semantic understanding through embeddings captures meaning beyond exact keyword matches

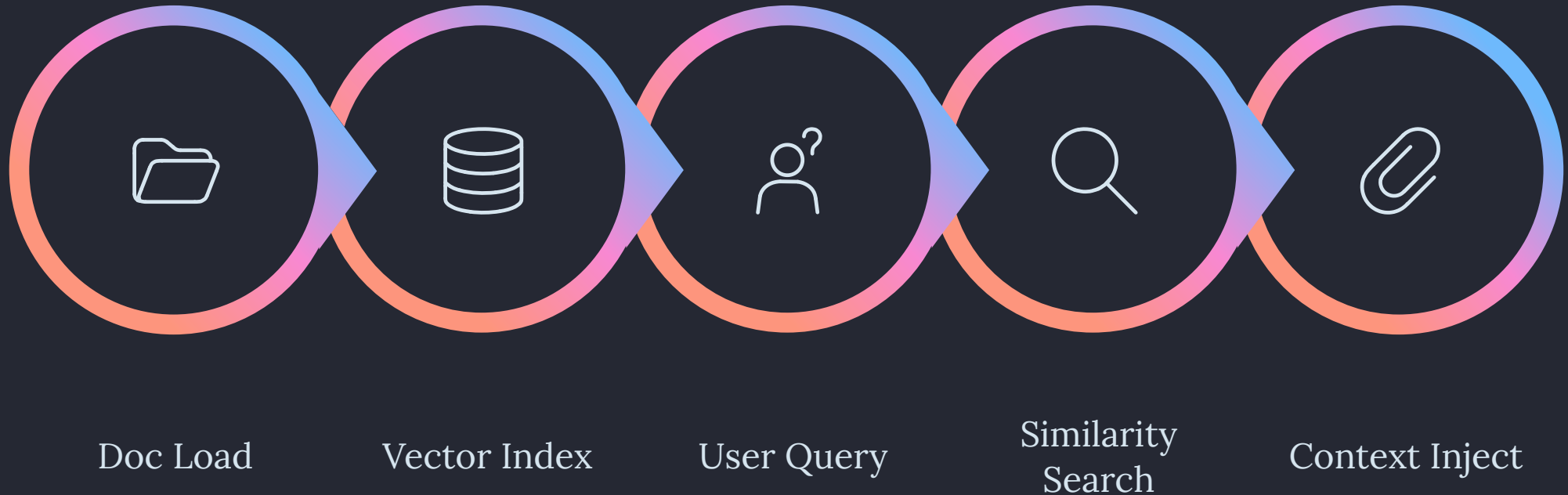
Keyword Search

Exact term matching ensures precision and handles specific entity queries

Hybrid Approach

Combine results using reciprocal rank fusion or weighted scoring for optimal relevance

RAG Chain Architecture



Integration with Gemini Models



Response Generation

Gemini models process retrieved context along with user queries to generate accurate, context-aware responses. The system handles prompt formatting and response parsing automatically.

Context Window Management

Truncate or compress retrieved documents to fit within Gemini's token limits while preserving critical information



Handling Context and Citations

01

Context Window Limits

Monitor token usage and implement dynamic truncation strategies

02

Prompt Engineering

Design templates that optimally structure retrieved documents and queries

03

Response Formatting

Generate structured outputs with clear source attribution

04

Citation Generation

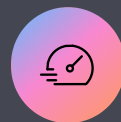
Include document references and metadata in responses for traceability

Key Takeaways



Robust Pipeline

LangChain simplifies document loading, preprocessing, and chunking



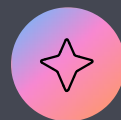
Efficient Retrieval

pgvector enables fast vector similarity search at scale



Hybrid Power

Combine vector and keyword search for optimal results



Gemini Integration

Generate context-aware responses with proper citations

Building production-ready RAG systems requires careful attention to chunking strategies, vector indexing, and prompt engineering. The combination of LangChain, PostgreSQL/pgvector, and Gemini provides a powerful foundation for enterprise applications.